



Multimodal RAG

for Arabic Document Retrieval

Mistral OCR + Jina v4 Embeddings + FAISS + Mistral LLM

 Project Documentation

The Problem

Why classical Arabic texts break standard RAG pipelines



Scanned PDFs

Source is image-based; standard text extractors return garbled output for Arabic right-to-left rendering.



Arabic Morphology

English-only embedders treat Arabic tokens as out-of-vocabulary, producing near-random retrieval results.



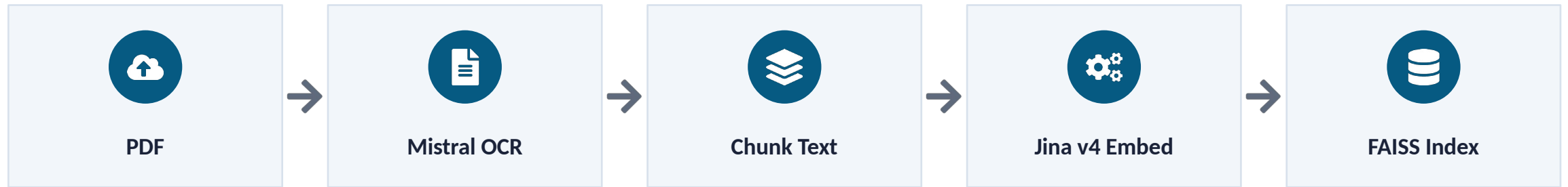
Verification Needed

Researchers must trace each retrieved verse back to the original printed page for academic integrity.

System Architecture

Two phases: offline indexing and online retrieval

Phase 1 — Indexing (offline)



Phase 2 — Retrieval (online)



Indexed metadata: source filename, page number, image path, and chunk text

Technology Stack

Each component chosen for Arabic-language support and ease of integration



Mistral OCR API

Cloud OCR with state-of-the-art Arabic support.



Jina Embeddings v4

Multilingual, multimodal embeddings; 2048-d vectors.



FAISS

Fast L2 similarity search over chunk vectors.



Mistral LLM

mistral-small-latest for grounded answer generation.



LangChain Splitter

Recursive character splitter with Arabic separators.



Gradio UI

Web interface with text snippets + page images.

Pipeline in Code

Three core scripts: ingest, retrieve, generate

ingest.py

```
ocr = MistralOCR()
pages = ocr.read_pdf(pdf_path)
imgs = rasterize_pdf_pages(pdf_path)
chunks = chunk_pages(pages, src, imgs)
vecs = jina.encode_text(
    texts=[c['text'] for c in chunks],
    prompt_name='passage')
faiss.write_index(index, 'text.faiss')
```

rag.py – search

```
def search(self, query, k=3):
    vec = self.embedder.encode_text(
        texts=[query],
        prompt_name='query')
    D, I = self.text_index.search(vec,
    k)
    return [self.text_chunks[i]
            for i in I[0]]
```

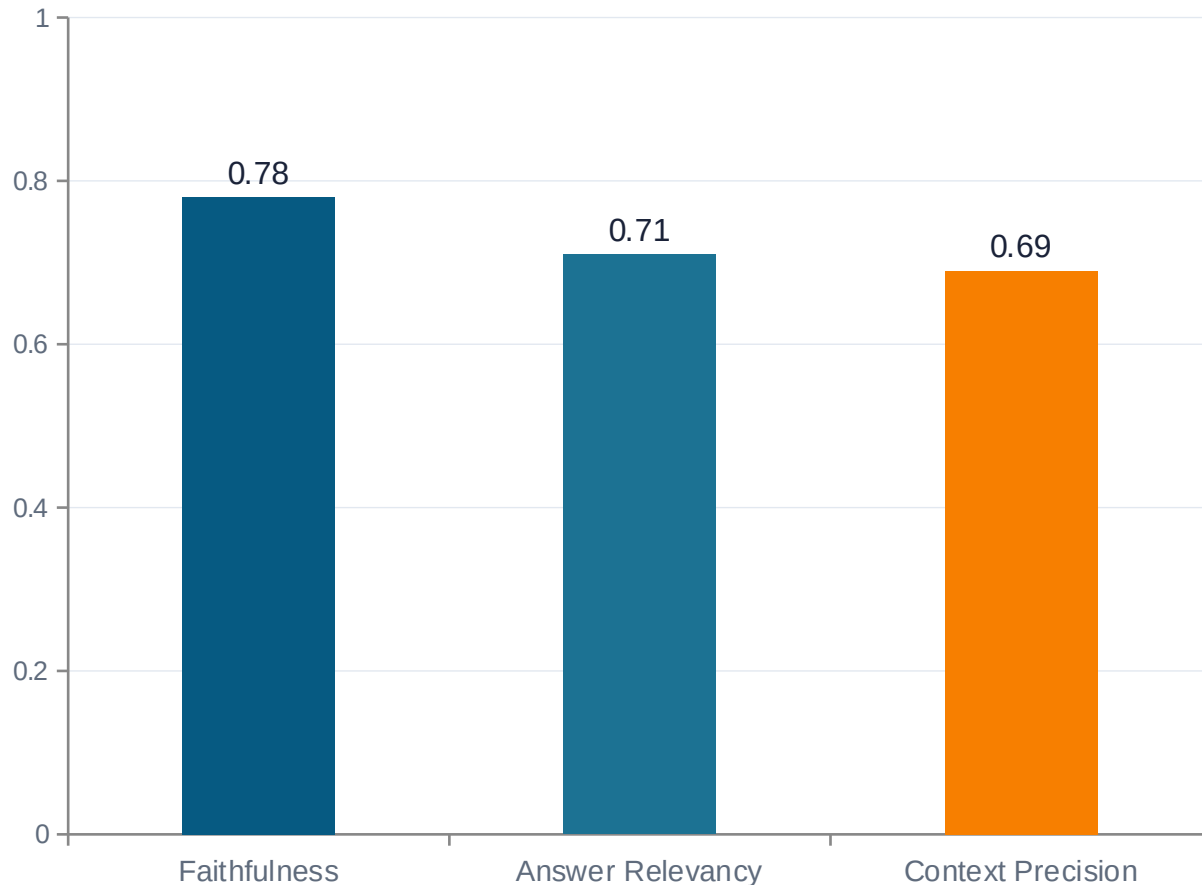
rag.py – answer

```
def answer(self, query, k=3):
    hits = self.search(query, k)
    ctx = '\n'.join(h['text']
                    for h in hits)
    prompt = build_prompt(query, ctx)
    return self.llm.chat.complete(
        model='mistral-small-latest',
        messages=[...])
```

Each chunk carries: source filename, page number, page-image path, and text

Evaluation

Reference-free similarity metrics computed with Jina v4 embeddings



Faithfulness — 0.78

Max similarity between answer and retrieved chunks; high = grounded answer.

Answer Relevancy — 0.71

Similarity between query and answer; high = on-topic response.

Context Precision — 0.69

Mean similarity of query vs retrieved chunks; high = correct retrieval.



Conclusion

Production-ready multilingual RAG built from open components

Achievements

- Clean Arabic OCR via Mistral API
- Multilingual retrieval with Jina v4
- Page images linked to every chunk
- Grounded, bilingual LLM answers
- Sub-second query latency on GPU

Future Work

- Add cross-encoder reranker for top-1 precision
- Enforce inline page citations in answers
- Swap to local OCR (dots.ocr) for cost
- Scale to multi-thousand page corpora
- Add user feedback loop for retrieval tuning

Thank you